Reporte de Práctica: Rendimiento de Flujos de Entrada y Salida con y sin Buffer en Java

1. Introducción y Objetivo

En la programación orientada a objetos con Java, las operaciones de entrada y salida (I/O) constituyen uno de los cuellos de botella más importantes para el rendimiento del sistema. Cada vez que nuestro código lee o escribe un dato directamente a través de un flujo sin buffer, se invoca una llamada al sistema operativo (system call) para acceder al disco físico o al almacenamiento. Estas interrupciones implican un cambio de contexto en la CPU y un costo de latencia muy elevado en comparación con las operaciones realizadas en la memoria RAM.

El objetivo de esta práctica es medir y comparar empíricamente el tiempo de ejecución que toman diversas clases de flujos de datos en Java (`java.io`) al procesar distintos volúmenes de información. Evaluaremos tanto flujos basados en caracteres (`Reader` / `Writer`) como en bytes (`InputStream` / `OutputStream`), comparando sus versiones directas e intermediadas por buffers. Para visualizar el comportamiento en diferentes escalas, los benchmarks se realizaron con volúmenes de datos de n = 100,000, n = 1,000,000 y n = 10,000,000 de elementos, analizando el impacto real que tiene el buffering en la eficiencia del software.

2. Tablas de Resultados y Mediciones

A continuación se presentan los tiempos de ejecución cronometrados (en milisegundos) para cada clase de flujo, realizando un total de tres iteraciones por prueba para calcular el promedio representativo.

Tabla 1: Mediciones para n = 100,000 elementos

Flujo utilizado	Tamaño	Ejec. 1 (ms)	Ejec. 2 (ms)	Ejec. 3 (ms)	Promedio (ms)
FileWriter	100,000	134	124	103	120.33
BufferedWriter	100,000	64	42	40	48.67

Flujo utilizado	Tamaño	Ejec. 1 (ms)	Ejec. 2 (ms)	Ejec. 3 (ms)	Promedio (ms)
FileReader	100,000	202	250	231	227.67
BufferedReader	100,000	31	35	25	30.33
FileOutputStream	100,000	1422	1796	809	1342.33
BufferedOutputStream	100,000	90	107	77	91.33
FileInputStream	100,000	2967	3876	2661	3168.00
BufferedInputStream	100,000	65	74	54	64.33

Tabla 2: Mediciones para n = 1,000,000 elementos

Flujo utilizado	Tamaño	Ejec. 1 (ms)	Ejec. 2 (ms)	Ejec. 3 (ms)	Promedio (ms)
FileWriter	1,000,000	725	578	553	618.67
BufferedWriter	1,000,000	597	501	426	508.00
FileReader	1,000,000	3012	2364	2284	2553.33
BufferedReader	1,000,000	98	101	336	178.33
FileOutputStream	1,000,000	11347	8428	7947	9240.67
BufferedOutputStream	1,000,000	600	583	643	608.67
FileInputStream	1,000,000	31554	31742	27595	30297.00
BufferedInputStream	1,000,000	485	130	650	421.67

Tabla 3: Mediciones para n = 10,000,000 elementos

Flujo utilizado	Tamaño	Ejec. 1 (ms)	Ejec. 2 (ms)	Ejec. 3 (ms)	Promedio (ms)
FileWriter	10,000,000	5261	5086	5511	5286.00
BufferedWriter	10,000,000	7323	5387	4434	5714.67
FileReader	10,000,000	20672	20047	20780	20499.67
BufferedReader	10,000,000	724	697	731	717.33
FileOutputStream	10,000,000	75464	77840	75031	76111.67
BufferedOutputStream	10,000,000	6438	4709	4961	5369.33

Flujo utilizado	Tamaño	Ejec. 1 (ms)	Ejec. 2 (ms)	Ejec. 3 (ms)	Promedio (ms)
FileInputStream	10,000,000	276762	285514	278520	280265.33
BufferedInputStream	10,000,000	7413	5319	7413	6478.67

3. Análisis y Respuestas al Cuestionario

1. ¿Cuál es la diferencia fundamental entre los flujos con buffer y sin buffer a nivel de sistema operativo?

La diferencia reside en cómo interactúan con el hardware y el sistema de archivos. Un flujo sin buffer (`FileInputStream`, `FileOutputStream`) realiza una llamada al sistema (system call) por cada byte o caracter procesado. Esto exige un cambio de modo (kernel/user space) constante, lo cual genera una sobrecarga tremenda en la CPU.

Por el contrario, los flujos con buffer (`BufferedInputStream`, `BufferedWriter`, etc.) mantienen un bloque de memoria en RAM. Cuando leemos, el buffer carga un bloque grande de bytes en una sola instrucción de disco; cuando escribimos, acumula los datos en RAM y solo realiza la escritura en el disco real cuando el buffer se llena (o al hacer `flush()`). Esto reduce radicalmente las llamadas al sistema.

2. ¿Cómo varía el rendimiento a medida que se incrementa el tamaño de n?

Al escalar de 100 mil a 10 millones de datos, el impacto se vuelve astronómico, especialmente en flujos binarios. Con FileInputStream, leer 10 millones de bytes tomó más de 280 segundos (~4.6 minutos) debido a los 10 millones de accesos individuales. En cambio, con `BufferedInputStream`, esa misma tarea tomó solo 6.4 segundos.

Curiosamente, en el caso de la escritura de caracteres (`FileWriter` vs `BufferedWriter` para 10M), la diferencia fue mínima (5286 ms vs 5714 ms). Esto ocurre porque las clases de caracteres en Java moderno implementan cierto buffering interno de codificación de texto, reduciendo la brecha. Sin embargo, para la lectura (`FileReader` vs `BufferedReader`), el buffer sigue siendo más de 28 veces más rápido.

3. Diferencia entre flujos orientados a caracteres y flujos orientados a bytes

Los flujos de bytes (`InputStream` / `OutputStream`) transfieren datos binarios crudos tal cual, procesando bloque a bloque de 8 bits sin interpretar su contenido (ideal para imágenes, ejecutables, audio, etc.).

Los flujos de caracteres (`Reader` / `Writer`) trabajan sobre unidades de 16 bits (char en Java) y aplican automáticamente tablas de codificación (como UTF-8 o UTF-16). Convierten los bytes recibidos a texto interpretable por la JVM. Trabajar con bytes crudos para texto puede romper caracteres especiales si no se maneja la codificación adecuada.

4. ¿Cuál es el tamaño por defecto del buffer en Java y cómo se modifica?

El tamaño predeterminado del buffer interno en la mayoría de las clases de `java.io` (como `BufferedReader` o `BufferedInputStream`) es de **8192 bytes (8 KB)**.

Para modificar este valor por uno personalizado (por ejemplo, a 32 KB para un mejor rendimiento en archivos masivos), se pasa un entero como segundo argumento en el constructor:int bufferSize = 32 * 1024; // 32 KB

BufferedReader reader = new BufferedReader(new FileReader("archivo.txt"), bufferSize);

5. ¿Por qué existe el método `readLine()` en `BufferedReader` y no en `BufferedInputStream`?

El método `readLine()` busca secuencias de fin de línea (`\n`, `\r` o `\r\n`) para retornar un `String`. Esto solo tiene sentido semántico en **flujos de texto** (caracteres), representados por los `Reader`.

Un `BufferedInputStream` procesa datos binarios abstractos (donde el byte `0x0A` que representa `\n` podría ser simplemente parte de la transparencia de una imagen PNG o el tono de un archivo de audio). Interpretar saltos de línea en un flujo de bytes podría corromper la lectura de archivos binarios.

4. Conclusión Personal

Esta práctica me sirvió para entender de manera práctica por qué en materias de sistemas operativos y programación avanzada se insiste tanto en evitar las llamadas al sistema innecesarias. Viendo los números fríos, es impresionante cómo una simple mala elección de clase al leer un archivo grande puede hacer que un programa tarde 4.5 minutos en lugar de 6 segundos.

El uso de buffers no es solo un adorno o "buena práctica", es una necesidad básica para que las aplicaciones sean escalables. De ahora en adelante, para cualquier desarrollo real en Java que involucre archivos o comunicación en red, usaré wrappers como `BufferedReader` o `BufferedOutputStream` por defecto, e incluso consideraría ajustar el tamaño del buffer si manejo grandes volúmenes de información en paralelo.